

Решеточные газы и решеточное уравнение Больцмана

Этап №3

Ромицына Анастасия

2 марта 2026

Российский университет дружбы народов

Информация

- Ромицына Анастасия Романовна
- Спелов Андрей Николаевич
- Исупов Олег
- Ко Антон



Вводная часть

- Изучить методы решеточных газов (LGA) и решеточного уравнения Больцмана (LBE) для моделирования гидродинамических течений.

Зачем нужны решеточные методы?

- Проблематика классического CFD и альтернатива
- Классика (Навье-Стокс, FEM/FVM): Сложная геометрия, ресурсоемкие сетки, трудности с распараллеливанием.
- Идея кинетического подхода: Моделируем не макро-параметры (давление, скорость), а динамику частиц на микроуровне.
- Наша цель:
- Показать путь от простейшего клеточного автомата (LGA) до современных фреймворков LBM с точки зрения кода и архитектуры.

Уровень 1: Клеточный автомат (LGA)

Модель NRP — простейшая программная модель

Структура данных:

Прямоугольная сетка.

Ячейка хранит биты: 0 или 1 (частица есть/нет).

Два шага эволюции (дискретный цикл):

Collision (Столкновение): Правила перераспределения частиц внутри ячейки при встрече лоб-в-лоб (битовый lookup table).

Streaming (Поток): Биты сдвигаются к соседям по направлениям скорости.

Недостатки LGA и переход к LBM

От дискретных частиц к функциям распределения

Почему LGA заменяют?

Высокий статистический шум.

Несоблюдение инвариантности Галилея.

Решение — LBM:

Оперируем не битами (0/1), а вероятностью (вещественные числа).

Программная особенность: Вместо побитовых операций — арифметика с плавающей точкой над массивами.

Основы решеточного уравнения Больцмана

Базовый алгоритм: Collision & Streaming

Дискретное уравнение (BGK-приближение): Расшифровка компонентов:

Перенос (Streaming): Левая часть уравнения. Данные летят в соседние ячейки.

Релаксация (Collision): Правая часть.

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) - f_i(\mathbf{x}, t) = -\frac{1}{\tau} [f_i(\mathbf{x}, t) - f_i^{eq}(\mathbf{x}, t)]$$

Расшифровка компонентов:

- **Перенос (Streaming):** Левая часть уравнения. Данные летят в соседние ячейки.
- **Релаксация (Collision):** Правая часть.
 - τ — время релаксации (связано с вязкостью жидкости).
 - f_i^{eq} — равновесное распределение Максвелла-Больцмана (локальное).

Локальное равновесие Равновесная функция — полином второго порядка:

$$f_i^{eq}(\rho, \mathbf{u}) = w_i \rho \left[1 + \frac{\mathbf{c}_i \cdot \mathbf{u}}{c_s^2} + \frac{(\mathbf{c}_i \cdot \mathbf{u})^2}{2c_s^4} - \frac{\mathbf{u} \cdot \mathbf{u}}{2c_s^2} \right]$$

Где:

- $\rho = \sum f_i$ — макроскопическая плотность.
- $\mathbf{u} = \frac{1}{\rho} \sum \mathbf{c}_i f_i$ — макроскопическая скорость.
- w_i — весовые коэффициенты решетки (например, для D2Q9: 4/9, 1/9, 1/36).
- c_s — скорость звука ($1/\sqrt{3}$ для стандартных решеток).

Архитектура программного комплекса LBM

Модульная структура солвера

Любой LBM-код состоит из блоков:

Инициализация: Задание

Цикл по времени:

Collision Kernel: Вычисление и обновление f (Вычислительно интенсивно, но локально).

Streaming Kernel: Обмен данными с соседними ячейками (Требует высокой пропускной способности памяти).

Граничные условия: Обработка «отскока» от стенок или входа потока.

Постобработка и I/O: Визуализация в ParaView / VTK.

Проблема Streaming (Потока) в памяти

Три подхода к реализации данных. Ключевая боль LBM — постоянное копирование массивов. Ping-Pong (A-B): Два массива. Плюс: Простой код. Минус: 50% времени занимает копирование памяти (memcpy). In-place (Сдвиг): Храним один массив. Меняем направления асинхронно, «прыгая» по памяти. Virtual Memory Shift (OpenLB): Используем виртуальную память (MREMAP на CPU / специализированные манипуляции на GPU). Физического копирования данных нет, меняются только указатели.

GPU (CUDA) и MPI

Использование GPU:

Идеальный массовый параллелизм: один тред GPU = один узел сетки.

Объединение Collision и Stream в одно ядро (fused kernel) для минимизации обращений к глобальной памяти. MPI для кластеров: Domain Decomposition:

Разрезаем область на кубы. Halo-обмен: На границах блоков обмениваемся слоями Проблема: Балансировка нагрузки, если часть GPU простаивает.

Эволюция программной реализации

LGA: Простые биты и логические правила (CUDA-автоматы).

Классический LBM: Массивы float/double, борьба за КЭШ и пропускную способность памяти (OpenLB, MPI + GPU).

Diff-LBM: Интеграция в графы вычислений PyTorch/JAX для решения обратных и оптимизационных задач «из коробки».

Перспектива: Гибридные квантово-классические алгоритмы.

Вывод: LBM прошел путь от игрушечных клеточных автоматов до промышленного инструмента CFD, а сейчас становится неотъемлемой частью AI-инжиниринга.

- Моделирование физических процессов и явлений на ПК / Д. А. Медведев, А. Л. Куперштох, Э. Р. Прууэл [и др.]. – Новосибирск : Новосиб. гос. ун-т, 2010. – 101 с. – ISBN 978-5-94356-933-3.